

MINI-PROJECT REPORT

AI-Powered Industrial Chatbot Using Hugging Face Large Language Models

Project Title	AI-Powered Industrial Chatbot Using Hugging Face Large Language Models
Program / Course	Industrial Training Program on Robotics and AI
Student Name	Ojas Sahu
Date	27 th September 2026

2. Introduction / Problem Statement

This project is an industrial chatbot that answers technical questions using a Hugging Face language model. Instead of relying only on the model's built-in knowledge, the chatbot retrieves relevant text from a set of industrial documents and uses that text as context for generating an answer. This approach is known as Retrieval-Augmented Generation (RAG), and it helps the chatbot give grounded answers instead of guessing.

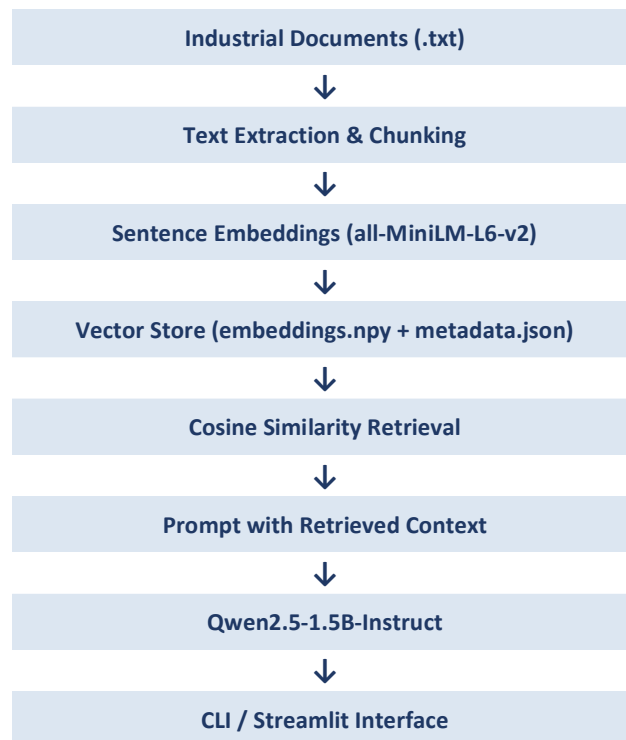
The assignment brief requires the chatbot to:

- Use a Hugging Face language model;
- Answer questions from provided industrial/technical documents;
- Apply prompt engineering or fine-tuning to improve domain-specific answers; and
- Provide a usable chatbot interface (CLI or web).

These requirements are met by combining Qwen2.5-1.5B-Instruct with a retrieval step built from sentence embeddings, using prompt engineering to ground the model's answers, and exposing the chatbot through a CLI and a Streamlit web app. The following sections address each of the six assignment questions in turn.

3. System Architecture (Question 1)

The system has two stages: an offline indexing stage that processes the documents once, and an online stage that runs every time a question is asked. The overall flow is shown below.



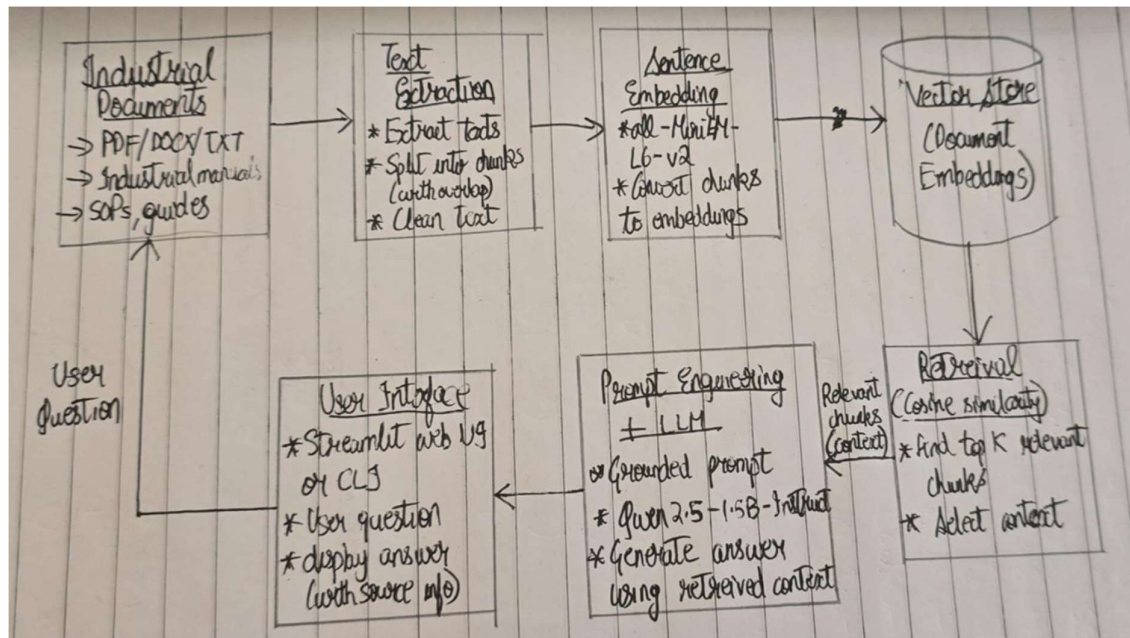


Figure 1. End-to-end architecture of the industrial chatbot.

build_index.py handles the offline stage: it reads the documents, splits them into chunks, embeds each chunk, and saves the embeddings and metadata to the vector_store folder. rag.py handles the online stage: it embeds the user's question, compares it against the stored chunk embeddings using cosine similarity, and passes the best-matching chunks to Qwen2.5-1.5B-Instruct along with the question. Both cli.py and the Streamlit app call this same rag.py logic, so the underlying pipeline is identical for both interfaces.

4. Model Selection (Question 2)

Qwen/Qwen2.5-1.5B-Instruct was chosen as the generation model. It is an instruction-tuned model available through Hugging Face Transformers, which means it follows the system/user prompt format used in this project (including the instruction to refuse when there isn't enough context). At 1.5B parameters it is small enough to run on a normal laptop or a single GPU, which made it practical for this mini-project without needing paid APIs or cloud compute or any paid services required to run the project.

For retrieval, sentence-transformers/all-MiniLM-L6-v2 is used as the embedding model. It converts each document chunk and each question into a fixed-length vector, and it is small and fast enough to embed text on the fly. The embedding model and the generation model play separate roles: MiniLM only produces vectors for similarity search, while Qwen2.5 only generates the final answer text. No benchmarking was done and both models were chosen for compatibility, size, and suitability for the task rather than measured scores.

5. Knowledge Ingestion (Question 3)

Three example demonstration industrial documents were used as source: `industrial_robot_safety.txt`, `cnc_machine_operation.txt`, and `plc_maintenance.txt`. These are placed in the `data/` folder and processed by `build_index.py`, which:

- reads and cleans the text of each document;
- splits it into overlapping chunks (450 words per chunk, 80-word overlap) so context isn't lost at chunk boundaries;
- embeds each chunk with `all-MiniLM-L6-v2`; and
- saves the embeddings (`embeddings.npy`) and the chunk text/source metadata (`metadata.json`) to the `vector_store` folder.

The index is built by running:

```
python src/build_index.py
```

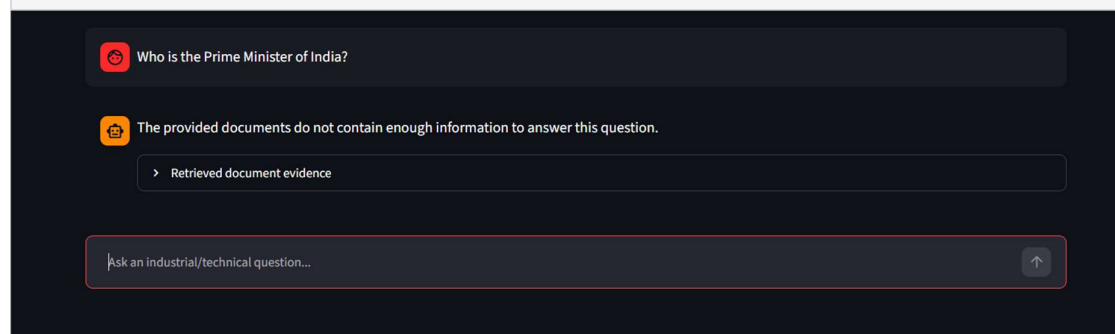
For the three documents used, this produced:

```
Indexed 3 chunks from 3 documents.
```

6. Prompt Engineering (Question 4)

This project uses prompt engineering, not fine-tuning. The weights of `Qwen2.5-1.5B-Instruct` are never changed. Instead, `build_prompt()` in `rag.py` builds a system prompt that tells the model to answer only from the retrieved document context, not to invent specifications or safety requirements, and to reply with a fixed fallback sentence when the context isn't enough: "The provided documents do not contain enough information to answer this." The prompt also asks the model to cite the source chunk it used (e.g. [Source 1]). A short, abridged version of the prompt looks like this:

```
System: Answer using only the supplied document context.  
       If information is insufficient, say so explicitly.  
       Cite sources as [Source N].  
  
User:  Document context: <retrieved chunks>  
       Question: <user question>
```



This is what makes the chatbot stay within the supplied documents instead of answering from general knowledge.

7. Q&A System Implementation (Question 5)

At runtime, `IndustrialRAG.answer()` in `rag.py` runs the following steps:

1. User asks a question (CLI or Streamlit).
2. The question is embedded using all-MiniLM-L6-v2.
3. Cosine similarity is used to compare the question embedding against all stored chunk embeddings.
4. The top-4 most similar chunks are retrieved as context.
5. The retrieved context and the question are placed into the prompt.
6. Qwen2.5-1.5B-Instruct generates the answer.
7. The answer, along with the retrieved source document(s), is shown to the user.

8. Chatbot Deployment (Question 6)

The chatbot is available through two interfaces, both built on the same `rag.py` backend.

8.1 CLI

```
python src/cli.py
```

The CLI prompts the user for a question, prints the generated answer, and lists the retrieved source document(s) with their similarity scores. Typing "exit" ends the session.

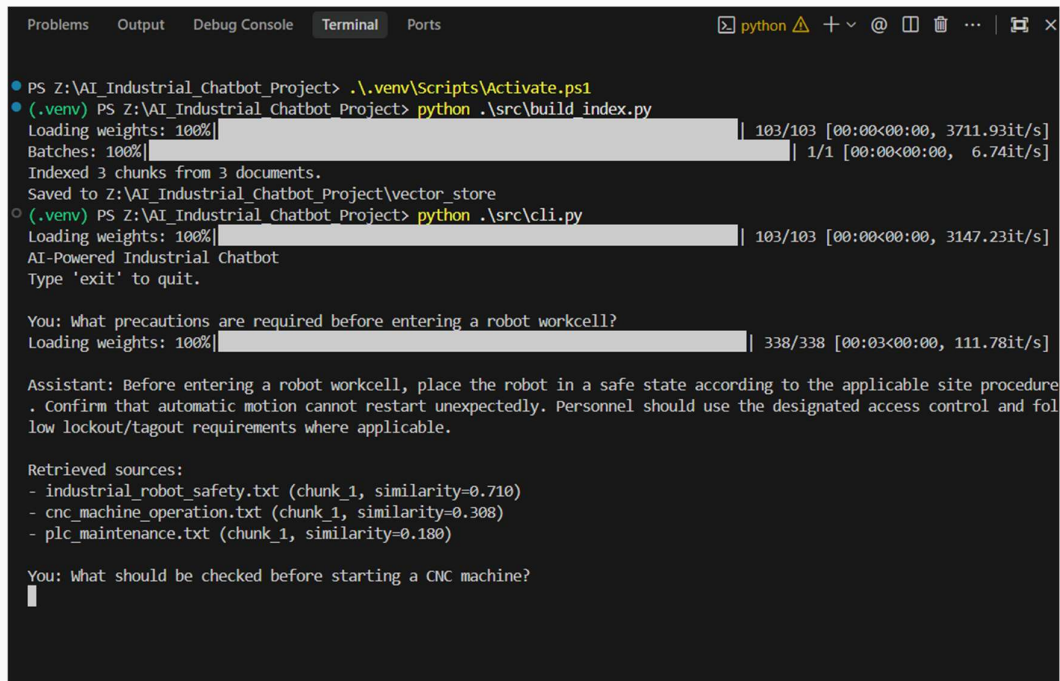


Figure 2. CLI demonstration.

8.2 Streamlit Web App

```
streamlit run app.py
```

This opens the chatbot in a browser, where the user types a question and gets the generated answer back, using the same retrieval-and-generation pipeline as the CLI.

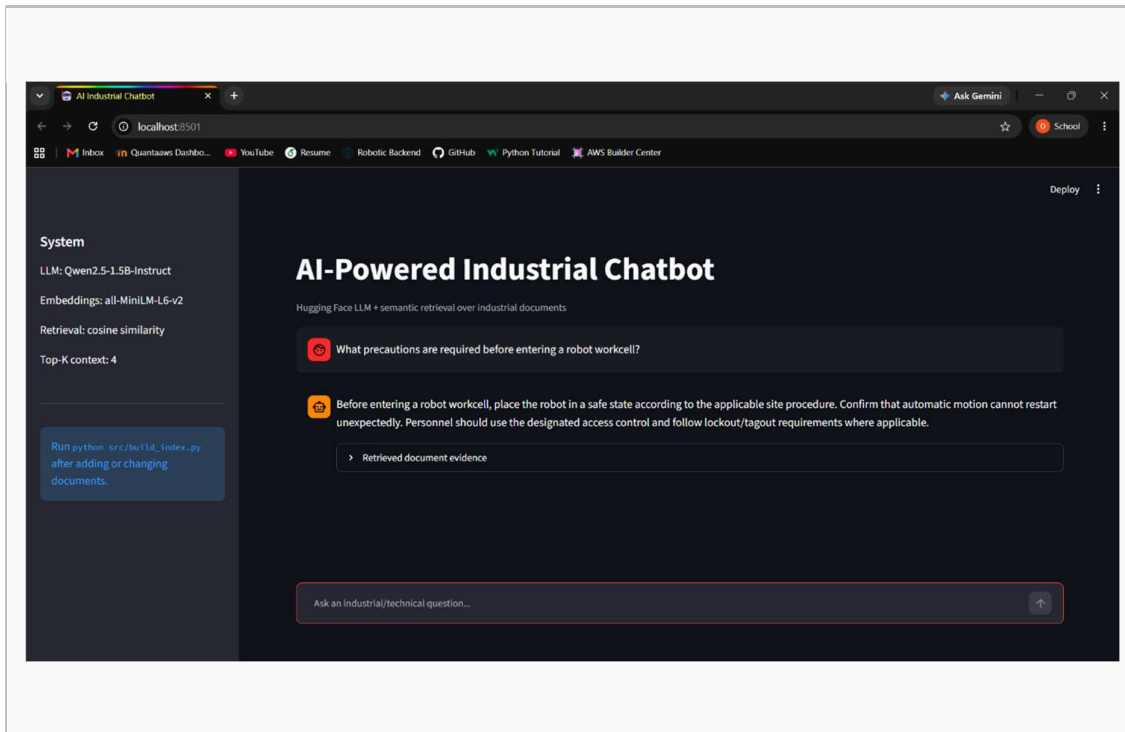


Figure 3. Streamlit chatbot interface.

9. Testing / Results

The chatbot was tested with the following questions through both interfaces:

Test	Question	Result
1	Precautions before entering a robot workcell	Correctly retrieved industrial_robot_safety.txt and gave a grounded safety answer.
2	Checks before starting a CNC machine	Correctly retrieved cnc_machine_operation.txt and listed the pre-start checks.
3	Symptoms of PLC communication failure	Correctly retrieved plc_maintenance.txt and listed the failure symptoms.
4	Maximum payload of the robot	Correctly stated the documents don't contain enough information.
5	Prime Minister of India	Correctly refused — question is outside the document knowledge base.

Tests 1–3 show the chatbot retrieving the right document and answering from it. Tests 4 and 5 show it correctly refusing to answer when the information isn't in the supplied documents.

10. Conclusion

The implemented chatbot satisfies all six assignment requirements: it has a clear retrieval-based architecture (Q1), uses a justified Hugging Face model pairing — Qwen2.5-1.5B-Instruct for generation and all-MiniLM-L6-v2 for retrieval (Q2), ingests the supplied documents into a chunked and embedded knowledge base (Q3), uses prompt engineering rather than fine-tuning to ground its answers (Q4), implements a full retrieval-plus-generation Q&A pipeline (Q5), and is deployed through both a CLI and a Streamlit web app (Q6). Testing confirmed the chatbot answers correctly from the supplied documents and refuses to answer questions outside their scope.

11. References

- Hugging Face. Transformers documentation. <https://huggingface.co/docs/transformers>
- Qwen Team, Alibaba Cloud. Qwen2.5-1.5B-Instruct model card. <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>
- Reimers, N. & Gurevych, I. Sentence-Transformers documentation. <https://www.sbert.net/>
- Streamlit Inc. Streamlit documentation. <https://docs.streamlit.io/>
- Python Software Foundation. Python documentation. <https://www.python.org/>